Documento de Diseño

Sistema de Gestión de Pedidos y Reseñas de Restaurantes

Proyecto 1 — CC3089 Base de Datos 2, Sección 20
Universidad del Valle de Guatemala
Catedrática: Daniela Mesalles

Nombre	Carné
Roberto Barreda	23354
Javier España	23361
Diego López	23747

Fecha de entrega: 10 de marzo de 2026

1. Descripción del caso de uso

El sistema modela una plataforma de delivery de comida similar a Uber Eats o PedidosYa, donde:

- Restaurantes publican su menú de artículos.
- Usuarios (clientes) realizan pedidos seleccionando artículos del menú.
- Cada Orden contiene uno o más artículos embebidos con cantidad, precio unitario y subtotal.
- Al entregar una orden, el usuario puede dejar una Reseña con calificación (1-5), comentario e imágenes almacenadas en GridFS.

El sistema demuestra las capacidades de MongoDB: documentos embebidos vs referenciados, transacciones, índices avanzados, GridFS, aggregation pipelines y operaciones bulk.

2. Arquitectura del sistema

Backend: Node.js + Express, driver nativo de MongoDB v6.13

Base de datos: MongoDB Atlas (replica set de 3 nodos)

Frontend: Vanilla JS SPA con dark theme, sin frameworks

Consola: Menú interactivo con readline-sync

El sistema ofrece dos modos de interacción: una API REST con 56 endpoints consumida por el frontend SPA, y un menú de consola interactivo para operaciones directas.

3. Modelo de datos

3.1 Colecciones

Colección	Documentos	Tipo	Validación
Restaurantes	25	Independiente con GeoJSON	JSON Schema (moderate)
Usuarios	200	Independiente	JSON Schema (moderate)
ArticulosMenu	~298	Referenciado (restaurante_id)	JSON Schema (moderate)
Ordenes	55,000	Embebido (items[]) + referenciado	JSON Schema (moderate)
Reseñas	~4,230	Referenciado + GridFS refs	JSON Schema (moderate)

Adicionalmente, GridFS crea automáticamente imagenes.files e imagenes.chunks.

```
Using MongoDB:          8.0.20
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

Atlas atlas-62wui6-shard-0 [primary] test> use delivery_db
switched to db delivery_db
Atlas atlas-62wui6-shard-0 [primary] delivery_db> show collections
ArticulosMenu
Ordenes
Resenas
Restaurantes
Usuarios
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

```
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Restaurantes.countDocuments()
25
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Usuarios.countDocuments()
200
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.ArticulosMenu.countDocuments()
278
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Ordenes.countDocuments()
55000
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Resenas.countDocuments()
4176
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

3.2 Relaciones

Documentos embebidos: Los items dentro de cada Orden son documentos embebidos. Cada item contiene un snapshot del nombre y precio del artículo al momento de la compra.

Documentos referenciados: Las relaciones entre colecciones usan ObjectId como referencia (usuario_id, restaurante_id, menu_item_id, orden_id).

```
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Ordenes.find().limit(1).pretty()
[
  {
    _id: ObjectId('69afcf41538a9ce2ceae7210'),
    usuario_id: ObjectId('69afcf40538a9ce2ceae7036'),
    restaurante_id: ObjectId('69afcf40538a9ce2ceae702f'),
    items: [
      {
        menu_item_id: ObjectId('69afcf40538a9ce2ceae71fa'),
        nombre: 'Empanadas',
        cantidad: 1,
        precio_unitario: 28.93,
        subtotal: 28.93
      },
      {
        menu_item_id: ObjectId('69afcf40538a9ce2ceae71f7'),
        nombre: 'Ensalada César',
        cantidad: 2,
        precio_unitario: 85.24,
        subtotal: 170.48
      },
      {
        menu_item_id: ObjectId('69afcf40538a9ce2ceae71f2'),
        nombre: 'Jugo de Naranja',
        cantidad: 2,
        precio_unitario: 117.91,
        subtotal: 235.82
      },
      {
        menu_item_id: ObjectId('69afcf40538a9ce2ceae71ee'),
        nombre: 'Ceviche Mixto',
        cantidad: 1,
        precio_unitario: 139.85,
        subtotal: 139.85
      }
    ],
    estado: 'entregado',
    total: 575.08,
    metodo_pago: 'efectivo',
    fecha_creacion: ISODate('2025-09-14T19:46:24.942Z'),
    fecha_actualizacion: ISODate('2025-09-14T19:46:24.942Z')
  }
]
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

3.3 Esquema: Restaurantes

```
{
  "_id": "ObjectId",
  "nombre": "string (requerido)",
  "descripcion": "string",
  "categoria": "enum [14 categorías]",
  "direccion": { "type": "Point", "coordinates": [lng, lat] },
  "telefono": "string",
  "rating_promedio": "double (0-5)",
  "total_resenas": "int (>= 0)",
  "activo": "bool (requerido)",
  "etiquetas": ["string"],
  "fecha_registro": "date"
}
```

3.4 Esquema: Usuarios

```
{
  "_id": "ObjectId",
  "nombre": "string (requerido)",
  "email": "string (requerido, único)",
  "password_hash": "string (requerido)",
```

```
  "rol": "enum [cliente, admin]",
  "fecha_registro": "date",
  "activo": "bool"
}
```

3.5 Esquema: ArticulosMenu

```
{
  "_id": "ObjectId",
  "restaurante_id": "ObjectId (ref → Restaurantes)",
  "nombre": "string (requerido)",
  "precio": "double (>= 0)",
  "categoria": "enum [8 categorías]",
  "disponible": "bool (requerido)",
  "stock": "int (>= 0)",
  "fecha_creacion": "date"
}
```

3.6 Esquema: Ordenes (shard key: _id)

```
{
  "_id": "ObjectId  ← SHARD KEY",
  "usuario_id": "ObjectId (ref → Usuarios)",
  "restaurante_id": "ObjectId (ref → Restaurantes)",
  "items": [{
    "menu_item_id": "ObjectId (ref → ArticulosMenu)",
    "nombre": "string (snapshot)",
    "cantidad": "int (>= 1)",
    "precio_unitario": "double",
    "subtotal": "double"
  }],
  "estado": "enum [pendiente, preparando, enviado, entregado, cancelado]",
  "total": "double",
  "metodo_pago": "enum [efectivo, tarjeta, transferencia]",
  "fecha_creacion": "date"
}
```

3.7 Esquema: Reseñas

```
{
  "_id": "ObjectId",
  "usuario_id": "ObjectId (ref → Usuarios)",
  "restaurante_id": "ObjectId (ref → Restaurantes)",
  "orden_id": "ObjectId (ref → Ordenes, único)",
  "calificacion": "int (1-5)",
  "comentario": "string (max 1000)",
  "imagenes": [{ "file_id": "ObjectId (GridFS)", "descripcion": "string" }],
  "fecha_creacion": "date"
}
```

4. Estrategia de índices

Se crearon 17 índices para optimizar todas las consultas. Todas usan IXSCAN (verificado con explain()).

#	Nombre	Colección	Tipo	Justificación
1	idx_restaurantes_geo	Restaurantes	2dsphere	Búsqueda geoespacial
2	idx_restaurantes_rating	Restaurantes	Compuesto	Ranking por calificación
3	idx_restaurantes_text	Restaurantes	Text	Búsqueda full-text
4	idx_restaurantes_categoria	Restaurantes	Simple	Filtro por categoría
5	idx_usuarios_email_unique	Usuarios	Unique	Unicidad de email
6	idx_menu_restaurante_stock	ArticulosMenu	Compuesto	Menú por restaurante + stock
7	idx_menu_restaurante_disponible	ArticulosMenu	Compuesto	Artículos disponibles
8	idx_menu_text	ArticulosMenu	Text	Búsqueda de artículos
9	idx_ordenes_usuario_fecha	Ordenes	Compuesto	Historial de usuario
10	idx_ordenes_restaurante_fecha	Ordenes	Compuesto	Órdenes por restaurante
11	idx_ordenes_restaurante_estado	Ordenes	Compuesto	Filtro estado + restaurante
12	idx_ordenes_estado	Ordenes	Simple	Consultas por estado
13	idx_ordenes_items_menu_item	Ordenes	Multikey	Platillos más vendidos
14	idx_resenas_restaurante_fecha	Reseñas	Compuesto	Reseñas por restaurante
15	idx_resenas_orden_unique	Reseñas	Unique	1 reseña por orden
16	idx_resenas_usuario_fecha	Reseñas	Compuesto	Reseñas por usuario
17	idx_resenas_imagenes_file	Reseñas	Multikey	Búsqueda por archivo GridFS

4.1 Validación con explain()

Todas las consultas fueron validadas con explain("executionStats") confirmando uso de IXSCAN:

```
Validando indices con explain()...

  [OK] Restaurantes cercanos (2dsphere)
       Scan: GEO_NEAR_2DSPHERE | Index: idx_restaurantes_geo
       nReturned: 5 | keysExamined: 13 | docsExamined: 7 | time: 0ms
  [OK] Restaurantes por categoria
       Scan: IXSCAN | Index: idx_restaurantes_categoria
       nReturned: 2 | keysExamined: 2 | docsExamined: 2 | time: 0ms
  [OK] Menu por restaurante + disponible
       Scan: IXSCAN | Index: idx_menu_restaurante_disponible
       nReturned: 0 | keysExamined: 0 | docsExamined: 0 | time: 0ms
  [OK] Ordenes por usuario + fecha
       Scan: IXSCAN | Index: idx_ordenes_usuario_fecha
       nReturned: 0 | keysExamined: 0 | docsExamined: 0 | time: 0ms
  [OK] Ordenes por estado
       Scan: IXSCAN | Index: idx_ordenes_estado
       nReturned: 11071 | keysExamined: 11071 | docsExamined: 11071 | time: 17ms
  [OK] Ordenes por restaurante + estado
       Scan: IXSCAN | Index: idx_ordenes_restaurante_estado
       nReturned: 0 | keysExamined: 0 | docsExamined: 0 | time: 0ms
  [OK] Resenas por restaurante + fecha
       Scan: IXSCAN | Index: idx_resenas_restaurante_fecha
       nReturned: 0 | keysExamined: 0 | docsExamined: 0 | time: 0ms
  [OK] Resenas por usuario + fecha
       Scan: IXSCAN | Index: idx_resenas_usuario_fecha
       nReturned: 0 | keysExamined: 0 | docsExamined: 0 | time: 0ms

Validacion de indices completada.

Conexión a MongoDB cerrada.

C:\Users\rjbar\OneDrive\Documents\GitHub\PRY1-BD>
```

```
C:\Users\rjbar\OneDrive\Documents\GitHub\PRY1-BD>cmd /c npm run indexes

> pry1-mongodb-delivery@1.0.0 indexes
> node src/scripts/createIndexes.js

Conectado a MongoDB: delivery_db

Creando índices...

  Restaurantes: idx_restaurantes_geo (2dsphere)
  Restaurantes: idx_restaurantes_rating (compuesto)
  Restaurantes: idx_restaurantes_text (texto)
  Restaurantes: idx_restaurantes_categoria (simple)
  Usuarios: idx_usuarios_email_unique (único)
  ArticulosMenu: idx_menu_restaurante_stock (compuesto)
  ArticulosMenu: idx_menu_restaurante_disponible (compuesto)
  ArticulosMenu: idx_menu_text (texto)
  Ordenes: idx_ordenes_usuario_fecha (compuesto)
  Ordenes: idx_ordenes_restaurante_fecha (compuesto)
  Ordenes: idx_ordenes_restaurante_estado (compuesto)
  Ordenes: idx_ordenes_estado (simple)
  Ordenes: idx_ordenes_items_menu_item (multikey)
  Resenas: idx_resenas_restaurante_fecha (compuesto)
  Resenas: idx_resenas_orden_unique (único)
  Resenas: idx_resenas_imagenes_file (multikey)
  Resenas: idx_resenas_usuario_fecha (compuesto)

Todos los índices creados exitosamente.
```

```
Atlas atlas-62wui6-shard-0 [primary] test> use delivery_db
switched to db delivery_db
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Ordenes.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { usuario_id: 1, fecha_creacion: -1 },
    name: 'idx_ordenes_usuario_fecha'
  },
  {
    v: 2,
    key: { restaurante_id: 1, fecha_creacion: -1 },
    name: 'idx_ordenes_restaurante_fecha'
  },
  {
    v: 2,
    key: { restaurante_id: 1, estado: 1, fecha_creacion: -1 },
    name: 'idx_ordenes_restaurante_estado'
  },
  { v: 2, key: { estado: 1 }, name: 'idx_ordenes_estado' },
  {
    v: 2,
    key: { 'items.menu_item_id': 1 },
    name: 'idx_ordenes_items_menu_item'
  }
]
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

4.2 notablescan

El sistema soporta activar notablescan para rechazar consultas que requieran COLLSCAN:

```
C:\Users\rjbar\OneDrive\Documents\GitHub\PRY1-BD>npm run notablescan:test

> pry1-mongodb-delivery@1.0.0 notablescan:test
> node src/scripts/configureDB.js test

Conectado a MongoDB: delivery_db

Probando que notablescan rechaza queries sin indice...

  [WARN] La consulta sin indice NO fue rechazada (notablescan puede no estar activo)
  [OK] Query CON indice (categoria) ejecutada sin problemas

Conexión a MongoDB cerrada.

C:\Users\rjbar\OneDrive\Documents\GitHub\PRY1-BD>
```

5. Estrategia de Shard Key

5.1 Shard key seleccionada

Campo: _id de la colección Ordenes (order_id)

Definido según indicaciones de la catedrática Daniela Mesalles.

5.2 Justificación

Criterio	Evaluación
Cardinalidad	Muy alta — cada orden tiene un ObjectId único, distribución granular entre shards
Frecuencia de escritura	ObjectId monotónicamente creciente; con hashed sharding se mitigan hot spots
Patrón de consulta	Consultas a órdenes individuales por _id son targeted queries directas
Distribución	Con hashed sharding sobre _id, distribución uniforme
Escalabilidad	Ordenes es la colección más grande (55,000+) y crece continuamente

5.3 Comando de sharding

```
sh.enableSharding("delivery_db")
sh.shardCollection("delivery_db.Ordenes", { _id: "hashed" })
```

5.4 Alternativa descartada

Se consideró restaurant_id pero se descartó por:

- Baja cardinalidad (solo 25 restaurantes) causa distribución desigual
- Restaurantes populares concentrarían demasiadas órdenes en un solo shard
- Consultas por _id individual serían scatter-gather en lugar de targeted

6. JSON Schema Validation

Todas las colecciones usan validación JSON Schema con nivel moderate y acción error:

- Los documentos existentes no se revalidan al modificar el esquema
- Los INSERT y UPDATE de documentos nuevos SÍ son validados
- Un documento que no cumpla el esquema causa un error de validación

Los tipos numéricos aceptan [double, int, long, decimal] para compatibilidad con la serialización del driver.

```
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Ordenes.findOne({ total: { $type: "double"
} })
{
  _id: ObjectId('69afcf41538a9ce2ceae7210'),
  usuario_id: ObjectId('69afcf40538a9ce2ceae7036'),
  restaurante_id: ObjectId('69afcf40538a9ce2ceae702f'),
  items: [
    {
      menu_item_id: ObjectId('69afcf40538a9ce2ceae71fa'),
      nombre: 'Empanadas',
      cantidad: 1,
      precio_unitario: 28.93,
      subtotal: 28.93
    },
    {
      menu_item_id: ObjectId('69afcf40538a9ce2ceae71f7'),
      nombre: 'Ensalada César',
      cantidad: 2,
      precio_unitario: 85.24,
      subtotal: 170.48
    },
    {
      menu_item_id: ObjectId('69afcf40538a9ce2ceae71f2'),
      nombre: 'Jugo de Naranja',
      cantidad: 2,
      precio_unitario: 117.91,
      subtotal: 235.82
    },
    {
      menu_item_id: ObjectId('69afcf40538a9ce2ceae71ee'),
      nombre: 'Ceviche Mixto',
      cantidad: 1,
      precio_unitario: 139.85,
      subtotal: 139.85
    }
  ],
  estado: 'entregado',
  total: 575.08,
  metodo_pago: 'efectivo',
  fecha_creacion: ISODate('2025-09-14T19:46:24.942Z'),
  fecha_actualizacion: ISODate('2025-09-14T19:46:24.942Z')
}
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

7. Transacciones multi-documento

La creación de órdenes usa una transacción multi-documento para garantizar atomicidad:

- 1. Iniciar sesión con client.startSession()
- 2. Iniciar transacción con session.startTransaction()
- 3. Verificar stock de cada artículo en ArticulosMenu
- 4. Decrementar stock con updateOne({ $inc: { stock: -cantidad } })
- 5. Insertar la orden en la colección Ordenes
- 6. Commit si todo exitoso, abort si stock insuficiente

```
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.ArticulosMenu.findOne({ stock: { $exists: t
true } })
{
  _id: ObjectId('69afcf40538a9ce2ceae70fa'),
  restaurante_id: ObjectId('69afcf40538a9ce2ceae7019'),
  nombre: 'Cheesecake',
  descripcion: 'Delicioso Cheesecake preparado con ingredientes frescos',
  precio: 106.22,
  categoria: 'Ensalada',
  disponible: true,
  stock: 268,
  fecha_creacion: ISODate('2025-11-17T23:51:48.124Z')
}
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

```
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Ordenes.distinct("estado")
[ 'cancelado', 'entregado', 'enviado', 'pendiente', 'preparando' ]
Atlas atlas-62wui6-shard-0 [primary] delivery_db>
```

8. GridFS

Bucket: imagenes (colecciones: imagenes.files, imagenes.chunks)

Operaciones implementadas:

- Subir: POST /api/resenas/:id/imagenes (multipart/form-data)
- Descargar: GET /api/resenas/archivos/:fileId
- Listar: GET /api/resenas/archivos
- Eliminar: DELETE /api/resenas/:id/imagenes/:fileId

```
═══════════════════════════════════════════════════
  RESENAS
═══════════════════════════════════════════════════

  1. Crear reseña
  2. Listar reseñas
  3. Reseñas de un restaurante
  4. Reseñas de un usuario
  5. Buscar por ID
  6. Subir imagen a reseña (GridFS)
  7. Listar archivos GridFS
  8. Eliminar imagen de reseña
  9. Actualizar reseña
  10. Eliminar reseña
  0. Volver

  Selecciona: 6

  ── Subir Imagen a Reseña (GridFS) ──
  ID de la rese├a: 69afcf4e538a9ce2ceaf48e8
  Ruta del archivo de imagen: C:\Users\rjbar\UVG.png
  Descripci├│n de la imagen: Made in UVG

  Imagen subida: UVG.png (ID: 69afd9784a39538c863b467d)

  Presiona ENTER para continuar...
```

ADD DATA ▾ | UPDATE | DELETE | EXPORT CODE

```
_id: ObjectId('69afcf4e538a9ce2ceaf48e8')
usuario_id : ObjectId('69afcf40538a9ce2ceae7036')
restaurante_id : ObjectId('69afcf40538a9ce2ceae702f')
orden_id : ObjectId('69afcf41538a9ce2ceae7210')
calificacion : 3
comentario : "Regular, esperaba un poco más."
▾ imagenes : Array (1)
  ▾ 0: Object
      file_id : ObjectId('69afd9784a39538c863b467d')
      descripcion : "Made in UVG"
fecha_creacion : 2025-09-14T21:46:24.942+00:00
```

9. Aggregation Pipelines

Se implementaron 6 pipelines analíticas accesibles vía API y consola:

9.1 Restaurantes mejor calificados

```
[ { $sort: { rating_promedio: -1, total_resenas: -1 } },
  { $limit: N },
  { $project: { nombre, categoria, rating_promedio, total_resenas } } ]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
1. Restaurantes mejor calificados
2. Platillos más vendidos
3. Ventas por restaurante
4. Ventas por período
5. Órdenes por estado
6. Distribución de calificaciones
0. Volver

Selecciona: 1

── Restaurantes Mejor Calificados (Aggregation Pipeline) ──
Top N [10]: 10
1. Antojitos GT | 3.21 | 145 reseñas | Americana
2. La Cocina de Mamá | 3.2 | 186 reseñas | Italiana
3. Pizzería Napoli | 3.16 | 174 reseñas | Peruana
4. Pollo Rico | 3.1 | 179 reseñas | Peruana
5. Burger Palace | 3.1 | 157 reseñas | Japonesa
6. Café Central | 3.08 | 167 reseñas | China
7. Sabores del Mundo | 3.08 | 160 reseñas | Mexicana
8. Sushi Zen | 3.06 | 177 reseñas | Francesa
9. Mariscos La Costa | 3.05 | 159 reseñas | Postres
10. Restaurant Chen | 3.03 | 167 reseñas | Guatemalteca

Presiona ENTER para continuar...
```

9.2 Platillos más vendidos

```
[ { $unwind: "$items" },
  { $group: { _id: "$items.menu_item_id", total: { $sum: "$items.cantidad" } } },
  { $sort: { total: -1 } }, { $limit: N } ]
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

═══════════════════════════════════════════════════════

  1. Restaurantes mejor calificados
  2. Platillos más vendidos
  3. Ventas por restaurante
  4. Ventas por período
  5. Órdenes por estado
  6. Distribución de calificaciones
  0. Volver

  Selecciona: 2

  ── Platillos Más Vendidos (Aggregation Pipeline con $unwind) ──
  Top N [10]: 10
  1. Empanadas | Vendidos: 1480 | Monto: Q102046 | Órdenes: 749
  2. Flan de Caramelo | Vendidos: 1459 | Monto: Q59833.59 | Órdenes: 724
  3. Alitas BBQ | Vendidos: 1446 | Monto: Q88393.98 | Órdenes: 722
  4. Hamburguesa Clásica | Vendidos: 1442 | Monto: Q66187.8 | Órdenes: 721
  5. Spring Rolls | Vendidos: 1432 | Monto: Q70483.04 | Órdenes: 705
  6. Tacos al Pastor | Vendidos: 1412 | Monto: Q156026 | Órdenes: 715
  7. Smoothie de Frutas | Vendidos: 1408 | Monto: Q144686.08 | Órdenes: 684
  8. Sushi Roll | Vendidos: 1405 | Monto: Q131774.95 | Órdenes: 708
  9. Cheesecake | Vendidos: 1403 | Monto: Q43843.75 | Órdenes: 708
  10. Alitas BBQ | Vendidos: 1401 | Monto: Q107176.5 | Órdenes: 682

  Presiona ENTER para continuar...
```

9.3 Ventas por restaurante

```
[ { $group: { _id: "$restaurante_id", total_ventas: { $sum: "$total" } } },
  { $lookup: { from: "Restaurantes" } },
  { $sort: { total_ventas: -1 } } ]
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

═══════════════════════════════════════════════════════

  1. Restaurantes mejor calificados
  2. Platillos más vendidos
  3. Ventas por restaurante
  4. Ventas por período
  5. Órdenes por estado
  6. Distribución de calificaciones
  0. Volver

  Selecciona: 3

  ── Ventas por Restaurante (Aggregation Pipeline) ──
  Top N [10]: 10
  1. El Mesón | Total: Q1175838.21 | Órdenes: 2237 | Prom: Q525.63
  2. Le Petit Bistro | Total: Q1101112.07 | Órdenes: 2160 | Prom: Q509.77
  3. Rincón Gourmet | Total: Q1100778.74 | Órdenes: 2152 | Prom: Q511.51
  4. Steak House Elite | Total: Q1099547.74 | Órdenes: 2165 | Prom: Q507.87
  5. Pizzería Napoli | Total: Q1094262.8 | Órdenes: 2214 | Prom: Q494.25
  6. Thai Garden | Total: Q1046068.69 | Órdenes: 2242 | Prom: Q466.58
  7. Sabores del Mundo | Total: Q1030052.52 | Órdenes: 2214 | Prom: Q465.25
  8. Antojitos GT | Total: Q992564.42 | Órdenes: 2156 | Prom: Q460.37
  9. Dulce Tentación | Total: Q984056.37 | Órdenes: 2229 | Prom: Q441.48
  10. Pasta Fresca | Total: Q981694.07 | Órdenes: 2176 | Prom: Q451.15

  Presiona ENTER para continuar...
```

9.4 Ventas por período

Agrupa órdenes por día, semana o mes usando $dateToString:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

  Periodos: dia, semana, mes
  Per├¡odo: 1,1,1
  Fecha inicio (YYYY-MM-DD, vac├¡o = todas):
  Fecha fin (YYYY-MM-DD, vac├¡o = todas):
  1. 2026-03-10 | Total: Q38306.85 | Órdenes: 91
  2. 2026-03-09 | Total: Q131350.99 | Órdenes: 297
  3. 2026-03-08 | Total: Q151712.66 | Órdenes: 322
  4. 2026-03-07 | Total: Q136273.9 | Órdenes: 322
  5. 2026-03-06 | Total: Q118935.66 | Órdenes: 281
  6. 2026-03-05 | Total: Q126776.7 | Órdenes: 300
  7. 2026-03-04 | Total: Q131966.4 | Órdenes: 316
  8. 2026-03-03 | Total: Q130084.65 | Órdenes: 311
  9. 2026-03-02 | Total: Q121082.7 | Órdenes: 295
  10. 2026-03-01 | Total: Q116108.92 | Órdenes: 271
  11. 2026-02-28 | Total: Q146083.45 | Órdenes: 325
  12. 2026-02-27 | Total: Q130299.05 | Órdenes: 294
  13. 2026-02-26 | Total: Q138007.12 | Órdenes: 305
  14. 2026-02-25 | Total: Q131482.72 | Órdenes: 298
  15. 2026-02-24 | Total: Q127523.94 | Órdenes: 284
  16. 2026-02-23 | Total: Q134110.32 | Órdenes: 295
  17. 2026-02-22 | Total: Q129156.76 | Órdenes: 297
  18. 2026-02-21 | Total: Q129851.62 | Órdenes: 304
  19. 2026-02-20 | Total: Q125173.73 | Órdenes: 288
  20. 2026-02-19 | Total: Q145486.94 | Órdenes: 331

  Presiona ENTER para continuar...
```

9.5 Órdenes por estado

```
[ { $group: { _id: "$estado", count: { $sum: 1 } } },
  { $sort: { count: -1 } } ]
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

═══════════════════════════════════════════════════════
  ANALÍTICAS Y REPORTES
═══════════════════════════════════════════════════════

  1. Restaurantes mejor calificados
  2. Platillos más vendidos
  3. Ventas por restaurante
  4. Ventas por período
  5. Órdenes por estado
  6. Distribución de calificaciones
  0. Volver

  Selecciona: 5

  ── Órdenes por Estado ──
  pendiente: 11071
  cancelado: 11060
  preparando: 11029
  enviado: 10947
  entregado: 10893

  Presiona ENTER para continuar...
```

9.6 Distribución de calificaciones

```
[ { $match: { restaurante_id: ObjectId } },
  { $group: { _id: "$calificacion", count: { $sum: 1 } } },
  { $sort: { _id: 1 } } ]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
═══════════════════════════════════════════════════════════
  ANALÍTICAS Y REPORTES
═══════════════════════════════════════════════════════════

  1. Restaurantes mejor calificados
  2. Platillos más vendidos
  3. Ventas por restaurante
  4. Ventas por período
  5. Órdenes por estado
  6. Distribución de calificaciones
  0. Volver

  Selecciona: 6
  ID del restaurante: 69afcf40538a9ce2ceae7019
  1: ▇▇▇
```

10. Operaciones Bulk

Se implementó bulkWrite para ArticulosMenu, Ordenes y Restaurantes:

- Bulk insert restaurantes: Inserta múltiples restaurantes en una sola operación
- Bulk update órdenes: Actualiza estados de múltiples órdenes simultáneamente
- Bulk write artículos: Combinación de insert/update/delete en una llamada

```
═══════════════════════════════════════════════════
  OPERACIONES BULK
═══════════════════════════════════════════════════

  1. Bulk insert restaurantes
  2. Bulk update estados de órdenes
  0. Volver

  Selecciona: 1

  ── Bulk Insert Restaurantes ──
  Cantidad de restaurantes a crear [5]: 5
  Insertados: 5

  Presiona ENTER para continuar...
```

```
═══════════════════════════════════════════════════
  OPERACIONES BULK
═══════════════════════════════════════════════════

  1. Bulk insert restaurantes
  2. Bulk update estados de órdenes
  0. Volver

  Selecciona: 2

  ── Bulk Update Estados ──
  Esto actualizará las primeras N órdenes pendientes a preparando.
  Cantidad [10]: 10
  Actualizados: 10

  Presiona ENTER para continuar...
```

11. Operaciones CRUD y de arrays

11.1 Operaciones básicas

- insert: insertOne, insertMany
- find: find, findOne con filtros, proyecciones, sort, skip, limit
- update: updateOne, updateMany
- delete: deleteOne, deleteMany
- count: countDocuments
- distinct: distinct

11.2 Operaciones de arrays

- $push: Agregar item a una orden
- $pull: Eliminar item de una orden, eliminar imagen de reseña
- $addToSet: Agregar etiqueta única a un restaurante

11.3 Lookups

- $lookup para enriquecer órdenes con datos de usuario y restaurante
- $lookup para reseñas con datos de usuario

```
Atlas atlas-62wui6-shard-0 [primary] delivery_db> db.Ordenes.find({ estado: "entregado" }).expla
in("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'delivery_db.Ordenes',
    parsedQuery: { estado: { '$eq': 'entregado' } },
    indexFilterSet: false,
    queryHash: 'CB0BE68E',
    planCacheShapeHash: 'CB0BE68E',
    planCacheKey: '162F2C0B',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { estado: 1 },
        indexName: 'idx_ordenes_estado',
        isMultiKey: false,
        multiKeyPaths: { estado: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { estado: [ '["entregado", "entregado"]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10893,
    executionTimeMillis: 15,
    totalKeysExamined: 10893,
    totalDocsExamined: 10893,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 10893,
      executionTimeMillisEstimate: 10,
      works: 10894,
      advanced: 10893,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
```

```
      restoreState: 0,
      isEOF: 1,
      docsExamined: 10893,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 10893,
        executionTimeMillisEstimate: 10,
        works: 10894,
        advanced: 10893,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { estado: 1 },
        indexName: 'idx_ordenes_estado',
        isMultiKey: false,
        multiKeyPaths: { estado: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { estado: [ '["entregado", "entregado"]' ] },
        keysExamined: 10893,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: '9241EC584E430560AF4E83F9BAA7E9300F028D19CB596BEE6D3EC30A6F81B10D',
  command: {
    find: 'Ordenes',
    filter: { estado: 'entregado' },
    '$db': 'delivery_db'
  },
  serverInfo: {
    host: 'ac-xj00egl-shard-00-01.xnty1eh.mongodb.net',
    port: 27017,
    version: '8.0.20',
    gitVersion: '28927c60881a488fcbc5fd4d925b410f33258827'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 16793600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
```

12. API REST (56 endpoints)

Base URL: http://localhost:3000/api

Restaurantes (12 endpoints)

Método	Endpoint	Descripción
GET	/restaurantes	Listar (paginado)
POST	/restaurantes	Crear restaurante
GET	/restaurantes/cercanos	Búsqueda geoespacial
GET	/restaurantes/buscar	Búsqueda full-text
GET	/restaurantes/categorias	Categorías distintas
PATCH	/restaurantes/varios	updateMany
DELETE	/restaurantes/varios	deleteMany
GET	/restaurantes/:id	Obtener por ID
PUT	/restaurantes/:id	Actualizar
DELETE	/restaurantes/:id	Eliminar
POST	/restaurantes/:id/etiquetas	Agregar etiqueta ($addToSet)
DELETE	/restaurantes/:id/etiquetas/:etiqueta	Eliminar etiqueta

Usuarios (5 endpoints)

Método	Endpoint	Descripción
GET	/usuarios	Listar
POST	/usuarios	Crear
GET	/usuarios/:id	Obtener
PUT	/usuarios/:id	Actualizar
DELETE	/usuarios/:id	Eliminar

Artículos Menú (7 endpoints)

Método	Endpoint	Descripción
GET	/menu/restaurante/:id	Menú por restaurante
POST	/menu	Crear artículo
POST	/menu/varios	insertMany
GET	/menu/buscar	Búsqueda full-text
GET	/menu/:id	Obtener
PUT	/menu/:id	Actualizar
DELETE	/menu/:id	Eliminar

Órdenes (11 endpoints)

Método	Endpoint	Descripción
GET	/ordenes	Listar
POST	/ordenes	Crear con transacción
GET	/ordenes/usuario/:id	Por usuario
GET	/ordenes/restaurante/:id	Por restaurante
PATCH	/ordenes/varios	updateMany
DELETE	/ordenes/varios	deleteMany
GET	/ordenes/:id	Obtener con $lookup
PATCH	/ordenes/:id/estado	Actualizar estado
POST	/ordenes/:id/items	Agregar item ($push)

DELETE	/ordenes/:id/items/:itemId	Eliminar item ($pull)
DELETE	/ordenes/:id	Eliminar

Reseñas (11 endpoints)

Método	Endpoint	Descripción
GET	/reseñas	Listar
POST	/reseñas	Crear
GET	/reseñas/restaurante/:id	Por restaurante
GET	/reseñas/usuario/:id	Por usuario
GET	/reseñas/archivos	Listar archivos GridFS
GET	/reseñas/archivos/:fileId	Descargar archivo
GET	/reseñas/:id	Obtener
POST	/reseñas/:id/imagenes	Subir imagen (GridFS)
DELETE	/reseñas/:id/imagenes/:fileId	Eliminar imagen
PUT	/reseñas/:id	Actualizar
DELETE	/reseñas/:id	Eliminar

Analíticas (7 endpoints)

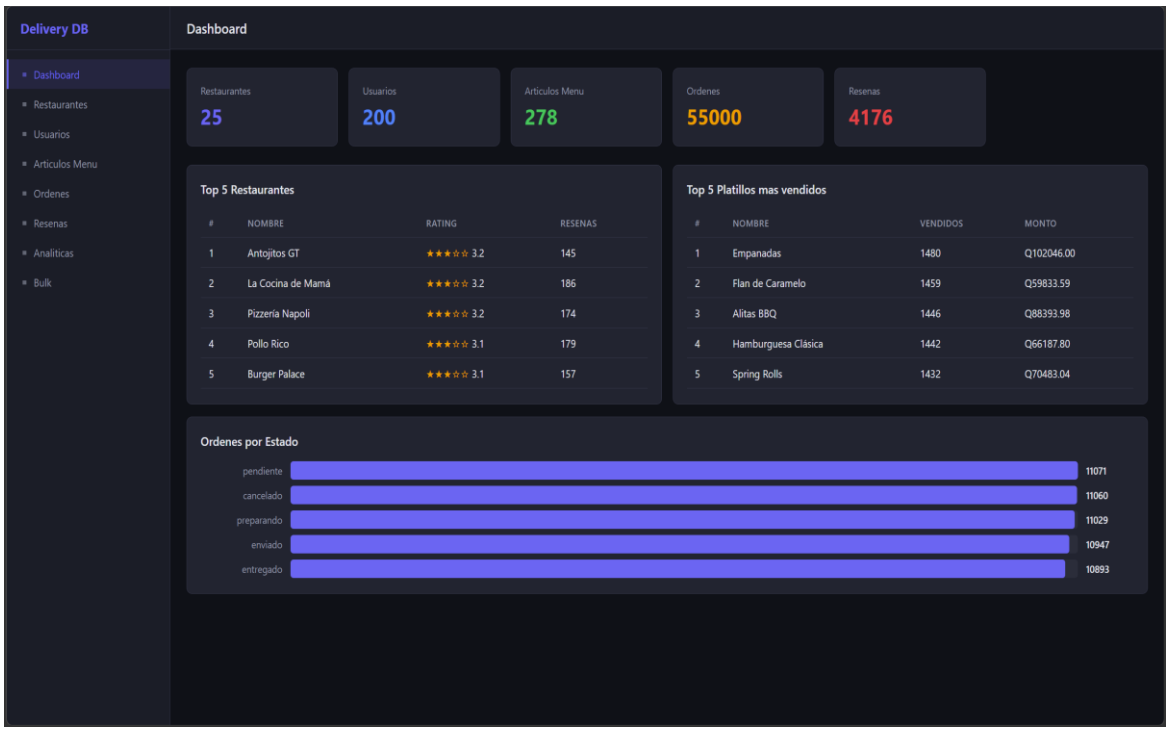
Método	Endpoint	Descripción
GET	/analiticas/resumen	Conteos generales
GET	/analiticas/restaurantes-mejor-calificados	Top calificación
GET	/analiticas/platillos-mas-vendidos	Más vendidos
GET	/analiticas/ventas-por-restaurante	Ventas agrupadas
GET	/analiticas/ventas-por-periodo	Ventas por período
GET	/analiticas/ordenes-por-estado	Conteo por estado
GET	/analiticas/distribucion-calificaciones/:id	Distribución

Bulk (3 endpoints)

Método	Endpoint	Descripción
POST	/bulk/articulos	bulkWrite ArticulosMenu
POST	/bulk/ordenes	bulkWrite Ordenes
POST	/bulk/restaurantes	Bulk insert restaurantes

13. Frontend SPA

Single Page Application con dark theme accesible en http://localhost:3000.

Características técnicas:

- Vanilla JS (sin frameworks)
- CSS custom properties para dark theme
- Navegación hash-based sin recarga de página
- Sistema de modales para CRUD
- Notificaciones toast
- Paginación en listas
- Gráficos de barras CSS para analíticas